

Terminal-RL oMLX System Architecture

2026-03-25

Contents

1	Terminal-RL — oMLX System Architecture	1
1.1	Stack Migration	1
1.2	System Overview	2
1.3	Component Map	3
1.3.1	Task Dataset	3
1.3.2	Async RL Loop (<code>train_async.py</code>)	3
1.3.3	oMLX Policy Slot (Rollout Inference)	3
1.3.4	CAMEL Agent (<code>agent/camel_agent.py</code>)	3
1.3.5	Router Server (<code>router_server.py</code> — port 18080)	3
1.3.6	Pool Server (<code>remote/pool_server.py</code> — port 18081)	4
1.3.7	Docker Containers (<code>remote/terminal_env.py</code>)	4
1.3.8	Rollout Buffer	4
1.3.9	oMLX PRM Slot (optional)	5
1.3.10	mlx-tune GRPOTrainer (Training)	5
1.4	Data Flow — One Episode	5
1.5	Memory Layout (Apple Silicon)	6
1.6	Key Configuration Knobs	6
1.7	Files Reference	7

1 Terminal-RL — oMLX System Architecture

RL training for a terminal/shell agent on **Apple Silicon** (MLX-native stack). The model learns to execute bash commands inside Docker containers to complete tasks, trained with GRPO (+ optional PRM step rewards).

Replaces: Slime → Ray → SGLang → Megatron-LM **With:** Async RL Loop | oMLX Policy Slot | oMLX PRM Slot | mlx-tune

1.1 Stack Migration

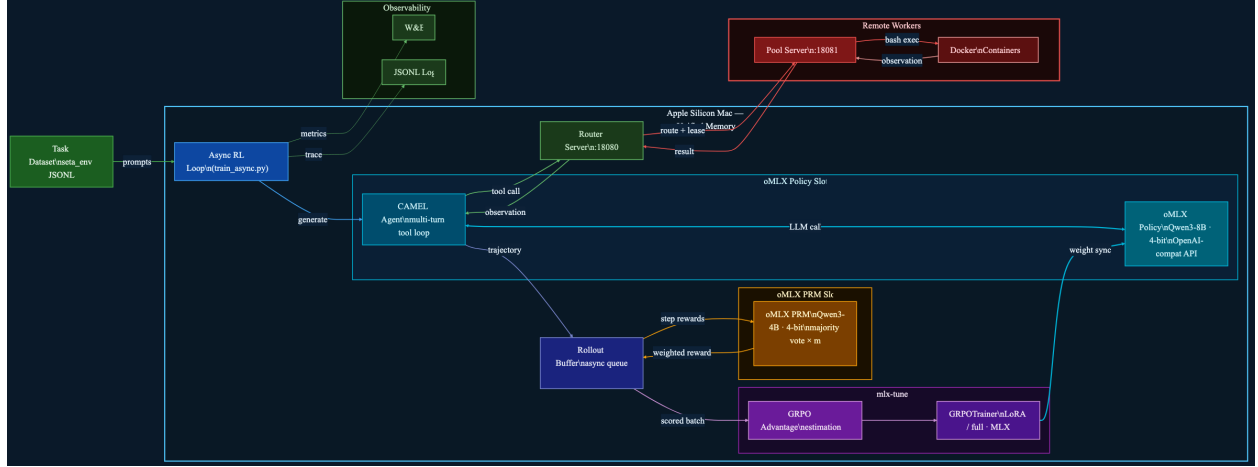


Figure 1: Architecture Diagram

Original (CUDA/Linux)	Replacement (MLX/macOS)	Purpose
Slime Orchestrator (Ray)	Async RL Loop (Python asyncio)	Training orchestration
SGLang Server	oMLX Policy Slot	Policy model inference
PRM SGLang Engine	oMLX PRM Slot	Step reward scoring
Megatron-LM Actor	mlx-tune GRPOTrainer	Gradient updates
Ray cluster	Unified Memory	Multi-process coordination

1.2 System Overview

The redesigned system runs entirely on a **single Apple Silicon Mac** using unified memory:

Tier	Machine	Role
Mac (Unified Memory)	1 Apple Silicon Mac (64–512 GB)	Async RL loop, oMLX inference (policy + PRM), mlx-tune training
Remote Workers	1–N CPU/Docker hosts	Pool servers managing Docker container environments

1.3 Component Map

1.3.1 Task Dataset

- **Source:** `seta_env` — from `camel-ai/seta-env`
 - **Format:** JSONL, each line has a `task` field (terminal instruction) and `score` field
 - **Prep:** `data_utils/download.py` → `data_utils/convert_task_to_dataset.py`
-

1.3.2 Async RL Loop (`train_async.py`)

- Replaces the Slime + Ray orchestrator
 - Reads prompts from dataset, dispatches rollouts to CAMEL Agent, collects scored batches
 - Calls `generate.generate` as the custom rollout function
 - Calls `rollout_log.rollout_log` to persist trajectory traces
 - Submits GRPO gradient updates to `mlx-tune` after each rollout batch
 - Logs metrics to W&B
-

1.3.3 oMLX Policy Slot (Rollout Inference)

- **Replaces:** SGLang Server
 - Serves the policy model (e.g. Qwen3-8B at 4-bit 5 GB) via OpenAI-compatible API
 - Runs natively on Apple Silicon using the MLX compute graph
 - **Features:** continuous batching, tiered KV cache (hot RAM → cold NVMe SSD), speculative decoding
 - Context length configurable (default 16384 tokens)
 - Weight sync from `mlx-tune` after each training step (in-place MLX array update)
-

1.3.4 CAMEL Agent (`agent/camel_agent.py`)

- Wraps CAMEL's `ChatAgent` with a custom backend that calls the **oMLX Policy Slot**
 - Runs a **multi-turn tool-call loop**:
 1. Gets context from memory
 2. Calls `oMLX /v1/chat/completions` for the next model response
 3. Parses tool calls (self-correcting JSON parser, up to `max_parse_errors=3`)
 4. Sends `exec_tool` requests to the Router Server
 5. Receives observations and adds them to memory
 6. Repeats until task complete or token budget exhausted
 - System prompt: `get_developer_agent_prompt` — Linux/Docker developer context
-

1.3.5 Router Server (`router_server.py` — port 18080)

- FastAPI server running on the **training machine**
- Sits between CAMEL Agent and remote worker pool servers
- **Endpoints:**

Endpoint	Purpose
POST /allocate	Reserve a Docker container for a task (returns <code>lease_id</code>)
POST /reset	Initialize container with task spec and return tool schemas
POST /exec_tool	Execute a bash command, return observation
POST /heartbeat	Keep lease alive
POST /evaluate	Score the completed episode
POST /close	Release the container
GET /healthz	Health check
GET /status	Worker pool status

- **Routing strategy:** SHA1 hash of `task_key` → consistent hash to primary worker, failover on error
- **Lease encoding:** "`{worker_idx}:{worker_lease_id}`" — encodes which worker holds the container

1.3.6 Pool Server (`remote/pool_server.py` — port 18081)

- FastAPI server running on **each remote worker machine**
- Manages a `WorkerPool` of `RunSlot` objects, each wrapping a `TerminalEnv` (Docker container)
- **Capacity:** `max_tasks=16`, `max_runs_per_task=8` per worker
- **Idle reaper:** background task cleans up slots idle > 600s
- **Idempotency cache:** 300s TTL prevents duplicate container allocation on retry

Container lifecycle per episode:

```

/allocate → lease_id
/reset    → user_msg + tool_schemas (pulls Docker image, starts container)
/exec_tool (× N turns) → bash output
/evaluate → float score (0.0-1.0)
/close    → releases container

```

1.3.7 Docker Containers (`remote/terminal_env.py`)

- Each `TerminalEnv` manages one Docker container per episode
- Container runs the task environment from `seta_env` task spec
- `exec_tool` maps to bash command execution inside the container
- `evaluate` runs the task-specific grader script inside the container
- Containers are isolated per episode — reset on each rollout

1.3.8 Rollout Buffer

- Python `asyncio.Queue` — replaces Ray's distributed object store

- Collects complete episode trajectories with final scores
 - Feeds into GRPO advantage estimation:

$$A_i = (r_i - \text{mean}(r)) / (\text{std}(r) + \epsilon)$$
over `n_samples_per_prompt=8` rollouts per task
 - `--dynamic_history` flag: GRPO uses full conversation history for advantage
-

1.3.9 oMLX PRM Slot (optional)

- **Replaces:** PRM SGLang Engine
 - Runs a separate **Judge / PRM model** (e.g. Qwen3-4B at 4-bit 3 GB) in a second oMLX model slot
 - Both policy and PRM slots live in unified memory — no cross-machine coordination
 - Scores **intermediate steps** (each bash turn), not just the final episode
 - `PRM_M=3` — 3 independent evaluations per step, majority vote
 - `PRM_STEP_COEF=1.0` — weight of step reward added to final reward
 - Can also call an external PRM endpoint via `PRM_OMLX_URL`
-

1.3.10 mlx-tune GRPOTrainer (Training)

- **Replaces:** Megatron-LM Actor
 - Runs GRPO gradient updates natively via the MLX compute graph
 - Supports **LoRA** (low-rank adapters, 8–16 GB) or **full fine-tuning**
 - No tensor parallelism needed — unified memory handles all sharding
 - KL penalty: `kl_loss_coef=0.01`, `kl_loss_type=k3`
 - Optimizer: AdamW, `LR=1e-6`
 - Weight sync back to oMLX Policy Slot via in-place MLX array update (no serialization)
-

1.4 Data Flow — One Episode

1. Async RL Loop picks task from dataset
2. CAMEL Agent starts multi-turn loop
3. Turn N:
 - a. Agent builds context (system + history)
 - b. oMLX Policy Slot generates response (bash command as tool call)
 - c. Agent sends `exec_tool` → Router → Pool Server → Docker
 - d. Docker runs bash command → stdout/stderr returned as observation
 - e. Agent appends observation to memory, starts Turn N+1
4. Episode ends (task complete / token budget / max turns)
5. `/evaluate` → Docker runs grader → score (0.0–1.0) returned
6. Full trajectory added to Rollout Buffer with score
7. After 8 trajectories for the same task:
 - a. oMLX PRM Slot scores each turn (optional)

- b. GRPO computes advantage across 8 rollouts
- c. mlx-tune runs gradient update (LoRA or full)
- d. Updated weights synced to oMLX Policy Slot (in-place)

1.5 Memory Layout (Apple Silicon)

Unified Memory (e.g. 64 GB)

oMLX Policy	oMLX PRM Slot
Qwen3-8B · 4bit	Qwen3-4B · 4bit
hot tier 5 GB	hot tier 3 GB

mlx-tune (LoRA adapters + gradients)
8-16 GB depending on batch size

oMLX SSD KV cache (cold tier → NVMe)
spills inactive KV blocks to disk

1.6 Key Configuration Knobs

Variable	Default	Effect
POLICY_MODEL	Qwen3-8B-4bit	Policy model loaded into oMLX Policy Slot
PRM_MODEL	Qwen3-4B-4bit	Judge model loaded into oMLX PRM Slot
CONTEXT_LEN	16384	Max context tokens per rollout turn
WORKER_URLS	—	Comma-separated pool server URLs
PRM_ENABLE	0	Enable oMLX PRM step scoring
PRM_M	3	Evaluations per step for majority vote
PRM_STEP_COEF	1.0	Weight of step reward vs final reward
n_samples_per_prompt	8	GRPO group size

Variable	Default	Effect
<code>rollout_batch_size</code>	16	Tasks per rollout round
<code>kl_loss_coef</code>	0.01	KL penalty vs base model
<code>lora_rank</code>	16	LoRA rank (0 = full fine-tune)

1.7 Files Reference

File	Role
<code>terminal_qwen3_8b_rl.sh</code>	Main training launcher (single node)
<code>terminal_qwen3_8b_prm_rl_2nodes.sh</code>	2-node variant with PRM enabled
<code>router_server.py</code>	Environment routing layer
<code>remote/pool_server.py</code>	Docker container pool manager
<code>remote/terminal_env.py</code>	Single Docker container wrapper
<code>agent/camel_agent.py</code>	CAMEL-based multi-turn agent loop
<code>agent/prm_agent.py</code>	PRM-aware agent variant
<code>agent/prompts.py</code>	System prompt: Linux developer persona
<code>generate.py</code>	Custom rollout function
<code>rollout_log.py</code>	JSONL trajectory logger
<code>inference_client.py</code>	oMLX turn client wrapper
<code>env_client.py</code>	HTTP client for Router/Pool API
<code>data_utils/download.py</code>	Dataset downloader (seta_env)
<code>data_utils/convert_task_to_dataset.py</code>	Task → JSONL converter